

图灵实战系列

Fullspace 实战

Capability-Manifold Agent Runtime

用向量检索取代图连线的下一代智能体编排

能力空间路由 · 运行时涌现 · 次线性扩展

基于 Fullspace 0.1.0 (Python 3.10+)
2026 年 · 教学版

前言

如果你用过 `LangGraph`，你一定写过这样的代码：先 `add_node` 注册节点，再用 `add_edge` 或 `add_conditional_edges` 把它们连成一张图，最后让一个路由器在每次分叉时逐个评估条件，决定下一步去哪。这套抽象已经服役了六十年——它的源头是 2010 年 Google 提出的 Pregel 整体同步模型，而 Pregel 又是对更古老的图计算传统的延续。

它好用，但封死了 agent 的上限。图的拓扑在编译期就冻结了，运行时无法生长；路由是离散枚举，条件边只能返回你预先声明的名字，既不能插值也不能优雅降级；更要命的是，每增加一个分支，每一步的路由成本就增加一次评估。

Fullspace 提出了一个激进的替代方案：**用能力空间路由取代边连线**。每个能力是高维语义流形中的一个点；“下一步去哪”不再由遍历一张连线图回答，而是由一次最近邻查询回答。你导航时看到的那个漂亮的 3D 球面，只是这个高维空间的人机投影——而真正的路由，永远发生在高维空间里。

本书的核心命题

用一次向量查询定位正确能力（而非预连 N 条边、每分支评估一次路由器），是 Fullspace 全部优势的单一来源。表达力、OOD 鲁棒性、规模化延迟、无屏障并行，都从这里派生。

本书适合谁

- 已经熟悉 `LangGraph` / `LangChain`，想理解“图之外”还有什么可能的工程师；
- 正在构建多步骤 LLM agent，被条件边的复杂度或无法动态扩展所困扰的开发者；

- 对向量检索、混合专家、稠密检索如何在编排中落地感兴趣的读者。 agent

如何阅读

第 1~2 章建立心智模型，建议顺序阅读。第 3 章是快速上手，跟着敲一遍代码。第 4~7 章是框架的核心机制（流形、引擎、流动策略、路由器），彼此紧密关联。第 8~10 章是状态、互操作与流式，可按需选读。第 11 章是基准测试，第 12 章讲如何扩展。书末三个附录分别是示例索引、API 速查与术语表。

本书所有 API 签名、行为描述均来自 Fullspace 0.1.0 真实源码，并标注了关键文件位置。代码清单大多可直接运行（HashEmbedder 无需任何 API key）。

目 录

本书适合谁	3
如何阅读	4
1.1 六十年前的抽象	9
1.2 图框架的三道天花板	9
1.3 能力空间路由：一次向量查询	10
1.4 两个决定一切的事实	11
1.5 小结	12
2.1 自顶向下看 Fullspace	13
2.2 模块地图	13
2.3 执行模型：闭环	14
2.4 四大延迟机制	15
2.5 小结	16
3.1 安装	17
3.2 第一个 agent	17
3.3 读懂轨迹	18
3.4 换上真实 embedding	19
3.5 3D 可视化	20
3.6 小结	20
4.1 Capability：流形上的一个点	21
4.2 Embedder：描述 → 向量	22

4.3 Distance: 度量邻近度	23
4.4 AnnIndex: 暴力 vs FAISS	24
4.5 Projector: 3D 投影 (只给人看)	25
4.6 Manifold: 把它们串起来	26
4.7 spawn-on-miss: 运行时涌现	27
4.8 小结	27
5.1 NodeResult: 三指令与优先级	29
5.2 coerce_result: handler 的自由度	30
5.3 四种执行入口	30
5.4 闭环每一步	31
5.5 状态合并: 每键 reducer	32
5.6 终止条件全集	32
5.7 小结	33
6.1 FlowPolicy 抽象	34
6.2 DiscreteFlow: 图等价	34
6.3 FieldFlow: 邻域与无屏障并行	35
6.4 WavefrontFlow: widening 邻域	36
6.5 _route_next 的分流	36
6.6 小结	37
7.1 三层结构	38
7.2 亲和度裁剪	38
7.3 歧义消解: 何时请 LLM	39

7.4 物化：运行时涌现	39
7.5 embedding 缓存	40
7.6 小结	40
8.1 每键 reducer 模型	41
8.2 Checkpoint：存什么	42
8.3 InMemory 与 Sqlite	42
8.4 resume 与时间旅行	43
8.5 trajectory：流形上的空间路径	44
8.6 小结	44
9.1 为什么互操作是承重墙	45
9.2 LG→FS：子图作为能力	45
9.3 FS→LG：engine 作为节点	46
9.4 FullspaceRunnable：langchain 契约	46
9.5 小结	47
10.1 StepEvent：每步一个事件	48
10.2 同步流	48
10.3 异步流与 async 节点	49
10.4 小结	49
11.1 harness 设计	50
11.2 六个对照用例	51
11.3 镜像模式：平手，但路由 FS 输	51
11.4 scaling：80~120× 的延迟优势	51

11.5 如何读懂这份基准	52
11.6 小结	52
12.1 自定义 Embedder	54
12.2 自定义 ANN 后端	55
12.3 自定义 FlowPolicy	56
12.4 自定义 Checkpointer	56
12.5 小结	57

第 1 章

从图到流形：Agent 编排的范式转移

要理解 Fullspace，先要理解它要取代什么，以及为什么要取代。这一章没有代码，只有三个观点和一个命题。

1.1 六十年前的抽象

“图”作为流程的抽象，可以追溯到 2010 年 Google 的 Pregel 论文。Pregel 把大规模图计算建模成一连串“超步”（superstep）：每个超步里所有节点并行执行，超步结束时有一个同步屏障（barrier），所有节点都完成才能进入下一个超步。LangGraph 继承了这个模型——它的节点是函数，边是转移规则，条件边是路由器，并行受 superstep 屏障约束。

这套抽象异常成功，但它把三件事焊死了：**拓扑在编译期确定、路由是离散枚举、并行受屏障约束。**

1.2 图框架的三道天花板

天花板	表现	后果
拓扑冻结	节点和边在 compile() 时固定	运行时无法新增/删除能力
离散枚举	条件边只能返回预声明的节点名	无法插值、无法对未见输入优雅降级
O(N) 路由	N 个专家就要 N 次分支评估	专家数线性增长时路由成本同步上升

表 1-1 图式 agent 框架的三道天花板

第一道天花板意味着：如果你的 agent 在运行中发现需要一个全新的能力（比如用户突然要算税），而你没有预先画出对应的节点和边，你就只能报错或硬塞进某个不合适的节点。

第二道天花板更隐蔽。条件边的本质是一个返回字符串的函数：

```
def router(state) -> str:
    if state["need"] == "calc":
        return "calc"
    elif state["need"] == "search":
        return "search"
    # 没有 default 分支？OOD 输入直接抛错
```

清单 1-1 LangGraph 条件边：离散枚举

它只能返回你写进 path mapping 的那几个名字。输入稍微偏离预期，路由器要么报错，要么需要你手写一个 catch-all。这就是为什么 LangGraph 对“分布外”（OOD）输入如此脆弱。

1.3 能力空间路由：一次向量查询

Fullspace 的替代方案出奇地简单：把每个能力描述嵌入成一个高维向量，放进一个度量空间；“下一步去哪”由一次最近邻查询回答。

```
manifold.register_many([
    Capability("calc", "perform arithmetic and math calculations"),
    Capability("search", "search the web for information"),
    Capability("end", "final answer output", metadata={"sink": True}),
])
# 路由 = 找最近邻, 没有任何 add_conditional_edges
res = engine.run("perform math calculations on numbers")
print(res.trajectory)  # ['calc', 'end']
```

清单 1-2 Fullspace: 没有 add_edge, 只有最近邻

注意这里没有一条 **add_edge**。任务文本 “perform math calculations” 被嵌入后，与所有能力描述比对，取最近邻——它落到了 calc

上。换一个任务文本，它就会落到别的能力上。**任务文本本身就是路由条件。**

单一来源

N 次条件分支评估 → 1 次最近邻查询。这一替换是 Fullspace 一切优势的根：它让路由从 $O(N)$ 变成可次线性的 $O(\log N)$ ，让拓扑得以在运行时涌现，让 OOD 输入自然回落到最近能力。

1.4 两个决定一切的事实

架构文档里有两句话，理解了它们就理解了 Fullspace 的全部设计：

事实一：图没有内在维度

把 N 个节点画在 2D 平面和画在球面上，是同一张图（完全图 K_N ）。所以“3D”要想有意义，几何必须承载计算语义。Fullspace 的基底是高维 embedding 流形（用于路由），3D 球面只是它的投影（用于人看）。路由从不使用投影。

事实二：能力空间路由胜过边连线

与其预连 N 条边、每分支评估一次路由器，不如用一次最近邻查询定位正确能力。这一个替换，就是全部优势的来源。

这两点解释了

Fullspace

所有看似“奇怪”的设计选择：为什么路由在高维空间做、为什么 3D 球只是装饰、为什么能力可以运行时涌现。后续章节会——落地到代码。

1.5 小结

图框架继承自

Pregel

的整体同步模型，带来了拓扑冻结、离散枚举、 $O(N)$

路由三道天花板。Fullspace 用“能力空间路由”取而代之：能力是高维流形上的点，路由是最近邻查询。这个单一替换，是后续所有章节的基础。下一章，我们从高空俯瞰 Fullspace 的整体架构。

第 2 章

全景：架构与模块地图

在动手之前，先建立一张完整的地图。这一章介绍 Fullspace 的模块划分、执行闭环，以及它在大规模时胜出的四大延迟机制。

2.1 自顶向下看 Fullspace

Fullspace 是一个纯 Python 库，核心仅依赖 NumPy。FAISS、sentence-transformers、UMAP、LangGraph 都是**可选扩展**——按需安装，框架自动启用。这意味着你 `pip install` 之后不需要任何 API key、不需要编译任何原生库，就能跑通完整的路由机制。

零依赖默认

Fullspace 的安装体验哲学：用户第一次安装不应被强拉重依赖。HashEmbedder（哈希特征）+ NumpyAnnIndex（暴力精确）+ PCAProjector（零依赖投影）构成开箱即用的默认三件套；要上生产再换 SentenceTransformersEmbedder / FaissIndex / UMAPProjector。

2.2 模块地图

包	职责	关键类型
manifold/	基底：描述→向量、ANN 索引、3D 投影	Manifold, Capability, Embedder, AnnIndex, Projector
engine/	闭环 + 流动策略 + 路由器 + 终止	Engine, Router, FlowPolicy, NodeResult, RunResult
state/	每键 reducer + 检查点（持久化/恢复/时间旅行）	merge_updates, Checkpointer, Checkpoint
interop/	双向 LangGraph 兼容（承重墙）	as_capability, as_langgraph_node, FullspaceRunnable
eval/	对照真实 LangGraph 的双轨基准 + FAISS scaling	run_all, scaling.run
viz/	3D 能力球可视化（HTML，无绘图依赖）	render_sphere

表 2-1 Fullspace 包结构与职责

依赖关系是清晰的漏斗形：底层 `types` / `distance` / `embedding` 各自独立，`manifold.py` 在顶端汇聚成门面；`engine` 消费 `manifold` 与 `state`；`interop` 单向依赖 `engine`，不改 `engine` 任何代码。一个关键事实是：**manifold 是完全被动的基底**——它提供数据结构和查询接口，但不驱动任何循环。驱动路由循环的是 `engine`。

2.3 执行模型：闭环

Fullspace 的执行是一个闭环。不管用同步的 `run`、流式的 `stream`，还是异步的 `ainvoke/astream`，背后都是同一个生成器驱动的循环：

```
task -> embed -> ANN 定位起点
-> 激活能力（流动策略决定激活几个）
    -> 运行 handler -> 合并状态更新（每键 reducer）
        -> 每个 handler 返回一个 intent 向量（下一步去哪）
            -> 检查点（若有 thread_id + checkpointer）
                -> 合并 intent -> ANN 路由到下一步
                    -> 命中 sink / halt / 预算即终止
```

清单 2-1 闭环七步

生成器驱动

同步路径 `_steps_sync` 是普通迭代器，异步路径 `_steps_async` 是异步迭代器。`run` 把生成器耗尽取最后一个事件，`stream` 把生成器直接交给调用者。这种设计让“阻塞/流式/异步/持久化”四套能力天然组合——这是最近一次重构（commit 4b2e8b7）的核心。

2.4 四大延迟机制

架构文档列出

在规模化时胜出的四个机制，前三个已经实现，第四个随真实集成落地：

Fullspace

LLM

机制	做法	代码位置
亲和力裁剪	一次 ANN 查询替代 N 次路由评估	engine/router.py
次线性 ANN	FAISS IVFFlat 在 $N=5k\sim 20k$ 比 $O(N)$ 快约 $80\sim 120\times$	manifold/index.py
无屏障并行	field/wavefront 激活邻域，无 superstep 屏障	engine/flow/
投机预热（计划中）	邻近前缀缓存，随真实 LLM 落地	—

表 2-2 Fullspace 的四大延迟机制

第一个机制解释了“为什么能力空间路由更快”，第二个解释了“为什么规模越大优势越大”，第三个解释了“为什么并行不需要屏障”。它们分别对应第 7、4、6 章。

2.5 小结

Fullspace 由 manifold（被动基底）、engine（主动闭环）、state（状态与持久化）、interop（生态互操作）四大块组成，eval 与 viz 是配套。执行是一个生成器驱动的七步闭环。规模化胜出的四大延迟机制里，前三个已实现。下一章，我们动手跑通第一个 agent。

第 3 章

快速上手

十分钟跑通你的第一个 Fullspace agent，读懂它的轨迹，然后换上真实 embedding。

3.1 安装

Fullspace 尚未发布到 PyPI，目前从 GitHub 安装。核心零重依赖，可选扩展按需安装：

```
pip install git+https://github.com/Muse2688/Fullspace.git # 核心
pip install faiss-cpu # + 次线性 ANN
pip install -e ". [langgraph, dev]" # 克隆后开发安装
```

清单 3-1 安装

extras 一览

embed-st (sentence-transformers)、embed-openai (openai)、ann-faiss (faiss-cpu)、proj-umap (umap-learn)、langgraph (langgraph + langchain-core)、dev (pytest + mypy)。

3.2 第一个 agent

下面这段代码来自官方快速开始，它用零依赖的 **HashEmbedder** 构建一个三能力流形，绑定纯函数 handler，然后运行：

```
from fullspace import Capability, HashEmbedder, Manifold
from fullspace.engine import Engine, NodeResult

manifold = Manifold(HashEmbedder())
manifold.register_many([
    Capability("search", "search the web for information"),
    Capability("calc", "perform arithmetic and math calculations"),
    Capability("summarize", "summarize a long document into key points"),
    Capability("end", "final answer output", metadata={"sink": True}),
])

agent = Engine(manifold)
agent.bind("search",
    lambda ctx: NodeResult(updates={"found": "..."},
                             intent="summarize a long document into key points"))
agent.bind("summarize",
    lambda ctx: NodeResult(updates={"summary": "..."}, goto="end"))
agent.bind("end",
    lambda ctx: NodeResult(updates={"answer": "..."}))

result = agent.run("search the web for information")
print(result.trajectory) # ['search', 'summarize', 'end']
```

清单 3-2 第一个 Fullspace agent

留意三点：第一，没有任何 **add_edge**；第二，search 用 **intent**（软路由）描述“下一步想做什么”，由 ANN 找最近邻；第三，summarize 用 **goto="end"**（硬路由）精确跳转，end 因为标记了 **sink=True** 而自然终止。

3.3 读懂轨迹

result 是一个 RunResult，关键字段如下：

字段	含义
state	最终合并后的共享状态 dict
trajectory	访问过的 capability id 列表
steps	执行的步数
terminated_by	终止原因字符串
final_capability	最后执行的能力 id
step_groups	每步激活的能力分组（并行时一组多个）

表 3-1 RunResult 字段

终止原因有 8 种

sink（命中汇点）、halt（强制停止）、no_intent（三指令皆空）、budget（预算用尽）、empty（流形为空）、no_handler（能力无 handler）、bad_goto（goto 指向不存在）、no_route（路由无结果）。

3.4 换上真实 embedding

把 HashEmbedder 换成 SentenceTransformersEmbedder，把纯函数 handler 换成 LLM 驱动的，就从“可运行机制”走向“生产级语义”：

```
from fullspace.manifold.embedding import SentenceTransformersEmbedder
# 需先 pip install -e ".[embed-st]"
m = Manifold(SentenceTransformersEmbedder("all-MiniLM-L6-v2")) # 384 维
```

清单 3-3 生产级语义 embedding

HashEmbedder 不懂同义词

HashEmbedder 只度量“字面 token 重叠度”——search 与 query 是不同 token，相似度为 0。它足够让框架自治、可测、可演示，且对路由逻辑零侵入，但不是真语义。生产请换神经 embedder。

3.5 3D 可视化

Fullspace 自带一个零绘图依赖的 3D 能力球可视化，渲染成自包含的 HTML：

```
python -m fullspace.viz      # → fullspace_sphere.html
```

清单 3-4 生成 3D 能力球

它会注册 7 个能力、跑一次工作流，然后把能力点和执行轨迹画在一个半透明球面上，用浏览器打开即可交互旋转。但请牢记本章最重要的一句话

球面只是投影，路由不用它

3D 球是给人看的地图；ANN

索引是在高维空间里真正找路的指南针。Router.route 只调 manifold.nearest（高维 cosine），完全不碰

project。这个不变量被写在多个 docstring 和一个专门的单元测试里。

3.6 小结

你已能跑通一个 Fullspace agent，读懂 trajectory 与 terminated_by，知道如何升级到真实 embedding 与 3D 可视化。下一章我们下沉到最底层的 manifold，看“能力点”是如何被嵌入、索引和查询的。

第 4 章

能力流形 Manifold：路由的基底

Manifold 是 Fullspace 的“土壤”。它把能力描述文本映射成高维向量，建立可查询的索引，并提供投影。这一章逐层拆解它的六个组件。

4.1 Capability：流形上的一个点

```
@dataclass
class Capability:
    id: str
    description: str
    metadata: dict[str, Any] = field(default_factory=dict)
    vector: Optional[np.ndarray] = None # 由 Manifold.register 写入

    @property
    def is_sink(self) -> bool:
        return bool(self.metadata.get("sink", False))
```

清单 4-1 Capability 数据类

终止是约定，不是类型

Capability 没有“终止节点”子类型。终止语义完全靠 metadata["sink"]=True 这个约定字段，is_sink 只是它的便捷读取器。对比 LangGraph 用特殊常量节点 END——Fullspace 把“类型二元论”降维成了一个语义标记。

注意 vector 默认是 None：“能力点”在未被 Manifold 接收之前，是没有空间位置的。位置由 register 通过 embedder 赋予。

4.2 Embedder：描述 → 向量

Embedder 是一个抽象基类，只要求实现 `embed(text) → (dim,)` 单位向量。开箱即用的 HashEmbedder 用的是**带符号特征哈希** (hashing trick)：

```
def embed(self, text: str) -> np.ndarray:
    vec = np.zeros(self.dim, dtype=np.float32)
    for tok in _tokenize(text):
        # [a-z0-9]+ 转小写
        digest = hashlib.md5(tok.encode()).digest()
        bucket = int.from_bytes(digest[:4], "little") % self.dim # 落哪个维
        sign = 1.0 if (digest[4] & 1) == 0 else -1.0 # +1 还是 -1
        vec[bucket] += sign
    norm = np.linalg.norm(vec)
    if norm > 0:
        vec /= norm
    return vec
```

清单 4-2 HashEmbedder 的核心

为什么特征哈希能产生“语义相似度”

相同 token 必落同桶、必同号 (md5 确定性) → 共享 token 会在同一维度叠加, $\cos$ 相似度上升; 不同 token 的桶号与符号近似均匀随机 → 不相关贡献近似抵消。于是两段文本的 cosine 近似它们 token 集合的重叠度。这是 Weinberger et al. 2009 的 hashing trick。

为什么用 md5 而非内建 hash

Python 内建 hash 对字符串默认带 PYTHONHASHSEED 随机化, 每次进程启动都不同, 会让同一文本跨进程得到不同向量, 破坏索引一致性。md5 是确定性的跨平台哈希, 保证可复现。

生产环境可选 `SentenceTransformersEmbedder` (默认 384 维) 或 `OpenAIEmbedder` (默认 1536 维)。两者都不在顶层包导出, 需从子模块显式 import——这是“零依赖默认”策略的体现。

4.3 Distance：度量邻近度

`distance.py` 是最纯的一层, 仅依赖 `numpy`, 被 `index` 与路由共用。核心函数包括:

函数	作用
<code>normalize(v)</code>	L2 归一化；零向量原样返回
<code>cosine(a, b)</code>	余弦相似度， $[-1, 1]$ ；零向量返回 0
<code>cosine_to_all(q, M)</code>	query 与矩阵每行的相似度，一次矩阵-向量乘
<code>affinity(score)</code>	$(score+1)/2$ ，把 $[-1,1]$ 线性映射到 $[0,1]$
<code>top_k(scores, k)</code>	前 k 大下标，降序

表 4-1 distance 模块函数

argpartition 而非 argsort

`top_k` 用 `np.argpartition` ($O(N)$ 选择算法) 先圈出前 k 名，再只对这 k 个排序 ($O(k \log k)$)。当 N 大、 k 小时 (路由场景 $N=\text{数千}$ 、 $k=5$)，把“找前 5 名”从 $O(N \log N)$ 降到 $O(N)$ 。

4.4 AnnIndex: 暴力 vs FAISS

`NumpyAnnIndex` 是参考实现，暴力精确。它有一个关键优化——**读多写少**的工作负载下的 dirty 矩阵缓存：

```
def add(self, id, vector):
    self._store[id] = vector
    self._dirty = True          # 不立即重建矩阵

def search(self, query, k=5):
    matrix, ids = self._matrix_and_ids()  # 仅当脏时才重建
    scores = cosine_to_all(query, matrix)
    return [(ids[i], float(scores[i])) for i in top_k(scores, k)]
```

清单 4-3 dirty 缓存：重建从 per-query 摊到 per-mutation

NumpyAnnIndex.search 不自己实现距离或排序，完全委托给 distance 层——换度量或换 top-k 算法是单点修改。这是分层架构的力量。

FaissIndex 是可选的 ANN 加速后端。它有一个容易忽略的精妙分支：

```
if n < self.nlist * 39:           # FAISS 官方训练阈值
    idx = faiss.IndexFlatIP(d)     # 小数据：精确暴力
else:
    idx = faiss.IndexIVFFlat(quantizer, d, self.nlist) # 大数据：倒排
    idx.train(matrix); idx.nprobe = self.nprobe
```

清单 4-4 FaissIndex 自适应索引选择

魔法数 nlist*39 的由来

FAISS 官方推荐 IVF 的训练样本数至少为 nlist*39（来自 FAISS 源码 clustering.h 的阈值）。少于这个数 train 质量很差，所以自动降级为 IndexFlatIP。由于 embedding 都是单位向量，cosine==内积，用 IndexFlatIP 省掉二次归一化。

FAISS 的删除难题

FAISS 不支持廉价任意删除，所以 remove 只是删字典+置脏，下一次 search 整体重建。代价是删除后第一次查询慢，后续快。这与 NumpyAnnIndex 的增量更新形成对照。

4.5 Projector: 3D 投影（只给人看）

Projector 是两阶段 API：先 fit 学习投影参数，再反复 project。默认 PCAProjector 用截断 SVD 形式的 PCA——对中心化数据矩阵做瘦 SVD，取前 3 个右奇异向量：

```
def fit(self, vectors):
    self._mean = vectors.mean(axis=0)
    centered = vectors - self._mean
    k = min(3, min(centered.shape))    # SVD 秩受双维限制
    _, _, vt = np.linalg.svd(centered, full_matrices=False)
    comps = vt[:k]
    if k < 3:                          # 秩不足时补零方向
        comps = np.vstack([comps, np.zeros((3 - k, centered.shape[1]))])
    self._components = comps
```

清单 4-5 PCAProjector 的 fit

API 契约的恒定性

无论注册了多少能力（哪怕 0 个或 2 个），project 永远返回长度 3 的数组：空矩阵返回 zeros(3)，秩不足时补零方向。这是把数学正确性（SVD 秩亏损）包装成稳定 API 的典范。

UMAPProjector 是可选的非线性备选，能更好保留局部邻域——对“在球面上找最近能力”的人类导航更友好。作者的取舍很明确：默认零依赖+确定性+全局结构（PCA），想要更好的局部结构再换 UMAP。

4.6 Manifold：把它们串起来

Manifold 是门面，把 embedder + index + projector 串成一个完整工作流。register 是“能力点获得空间位置的唯一入口”：

```
def register(self, capability):
    vector = self.embedder.embed(capability.description)
    capability.vector = vector    # 1. 赋位
    self._caps[capability.id] = capability    # 2. 存字典
    self.index.add(capability.id, vector)    # 3. 更新索引
    self._projection_dirty = True    # 4. 标记投影脏
    return capability
```

清单 4-6 register：三层状态同步

Python 真值陷阱

manifold.py 用 `index is None` 而非 `index or ...`。因为 `AnnIndex` 定义了 `__len__`，空 `index` 是 `falsy`——如果写 `or`，用户传的空自定义 `index` 子类会被默默替换。好的 Python 代码必须意识到 `__len__`/`__bool__` 对默认值表达式的影响。

查询走 `nearest`：它把 `index` 的 `(id, score)` 提升为 `Hit(capability, score)`，并防御性过滤掉索引与字典不一致的条目。`nearest_id` 是不构造 `Hit` 的轻量版。

4.7 spawn-on-miss：运行时涌现

`find_or_materialize` 是 manifold 的灵魂，实现三大路径：

```
def find_or_materialize(self, query, threshold=0.5, k=1, materializer=None):
    hits = self.nearest(query, k=k)
    if hits and hits[0].score >= threshold:
        return hits[0]  # 路径 A：亲和力裁剪
    if materializer is None:
        return hits[0] if hits else None  # 路径 B：优雅降级
    cap = materializer(desc, hits[0].score if hits else 0.0)  # 路径 C
    self.register(cap)  # 物化并注册
    return Hit(cap, 1.0)
```

清单 4-7 `find_or_materialize` 三大路径

涌现的几何表达

当一个 `intent` 没有匹配能力，系统不是“失败”，而是“创建一个新能力”——这就是涌现。`materializer` 是涌现的工厂。`LangGraph` 的图是开发者预先画好、固定的；`Fullspace` 的能力空间会随运行时涌现地增长。

4.8 小结

Manifold 是被动基底：Capability 是点，Embedder 给点赋位，Distance 度量邻近，AnnIndex 提供可次线性的查询，Projector 提供只给人看的 3D 投影，Manifold 门面把它们串起来并提供 spawn-on-miss 涌现。路由永远在高维空间做，3D 只是投影。下一章，我们看 engine 如何在这个基底上驱动闭环。

第 5 章

执行引擎 Engine：闭环

Engine 是 Fullspace 的心脏。它消费 manifold 这个基底，驱动“定位→激活→执行→合并→意图→路由→终止”的闭环。这一章拆解闭环的每一步。

5.1 NodeResult：三指令与优先级

handler 通过 NodeResult 同时表达“写什么状态”和“下一步去哪”。后者有三种互斥语义，优先级为 **halt > goto > intent**：

```
@dataclass
class NodeResult:
    updates: dict = field(default_factory=dict)  # 合并进状态
    intent: Optional[Union[str, np.ndarray]] = None  # 软路由
    goto: Optional[str] = None  # 硬路由
    halt: bool = False  # 强制终止
```

清单 5-1 NodeResult 数据类

字段	路由方式	对应 LangGraph
intent	软路由：字符串被 embed，ANN 找最近邻	语义条件边
goto	硬路由：精确跳指定 id	固定边
halt	立即终止	直接到达 END

表 5-1 三指令对比

三指令皆空 = 优雅终止

如果一个 handler 返回的 NodeResult 既无 goto 又无 intent（且未 halt），engine 按 no_intent 终止。这正是“把子图当汇点”的写法：什么都不返回，让运行自然结束。

5.2 coerce_result: handler 的自由度

coerce_result 把 handler 的三种合法返回统一成 NodeResult，这让写 handler 极其自由：

handler 返回	归一化为
None	NodeResult(halt=True) —— 视为到此为止
dict	NodeResult(updates=dict) —— 只写状态，自然 no_intent 终止
NodeResult	原样返回
其他	抛 TypeError

表 5-2 handler 返回值归一化

5.3 四种执行入口

Engine 提供同步/异步 × 阻塞/流式 四个入口，背后共享同一个生成器：

入口	模式	返回
run	同步阻塞	RunResult (取最后一个事件)
stream	同步流式	Iterator[StepEvent], 每步 yield
ainvoke	异步阻塞	RunResult
astream	异步流式	AsyncIterator[StepEvent]

表 5-3 四种入口对照

run = 把 stream 消费到末尾

四套入口共享同一个步循环。run 内部就是 `_collect_sync(_steps_sync(...))`，ainvoke 内部就是 `async for ev in astream` 取最后一个。这解释了为什么它们的语义完全一致。

5.4 闭环每一步

以 stream 为例，闭环每步做这些事：

```

active = self.flow.select(self.manifold, task)    # locate (首步用 task)
group = [h.capability.id for h in active]
for h in active:                                # run
    ctx = NodeContext(state, trajectory, step, task)
    result = coerce_result(handler(ctx))          # 容错归一
    state = merge_updates(state, result.updates, spec) # merge (每键 reducer)
    if result.intent is not None:
        intents.append((result.intent, h.score))  # 收集意图
        if result.halt: ...; if result.goto: gotos.append(...)
    step += 1
next_active = self._route_next(gotos, intents)   # route (见第 7 章)

```

清单 5-2 单步伪代码

同组内是顺序执行

field/wavefront 一次激活多个能力，但同组内的 handler 是顺序执行、顺序合并，而非真并发。语义上是“同组并行”（updates 在本步内合并，intent 加权合成下一 query），代码层无真线程。

5.5 状态合并：每键 reducer

状态合并借鉴 LangGraph 的 channel/reducer 模型，但简化为“按 key 选 reducer”。默认 overwrite（最后写入获胜），可选 add（列表拼接）、last_value（None 表示保留旧值）：

```
def merge_updates(state, updates, spec=None):
    spec = spec or {}
    for key, value in updates.items():
        reducer = spec.get(key, overwrite)  # 未声明键走 overwrite
        state[key] = reducer(state.get(key), value)
    return state
```

清单 5-3 merge_updates：只合并被写过的键

add reducer 支持标量起步

add 把标量起步也支持了：第一次 add(state, {"msgs": "hi"}) 直接得到 ["hi"]，无需先写空列表。这简化了“消息历史”这类通道的初始化，是相对 LangGraph Annotated[list, add] 的细节优化。

5.6 终止条件全集

engine 有 8 种终止原因，分布在不同位置触发：

原因	触发条件
sink	命中的能力 is_sink (metadata["sink"]=True)
halt	某 handler 返回 halt=True
no_intent	既无 goto 也无 intent
budget	步数超过 max_steps (默认 25)
empty	流形为空, 或首步 select 返回空
no_handler	能力 id 无绑定 handler
bad_goto	goto 指向不存在的能力
no_route	路由器返回 None

表 5-4 8 种终止原因

Terminator 当前是设计占位

Terminator.check 只判 halt/sink/no_intent, 且当前 runtime 闭环并未调用它——预算与 bad_goto/no_route 都由 runtime 自己判。Terminator 主要作为 max_steps 载体与给测试/eval 使用的设计占位。本书如实说明这一现状。

5.7 小结

Engine 通过 NodeResult 的三指令 (halt>goto>intent) 表达 “下一步去哪”, 用 coerce_result 给 handler 极大自由, 用每键 reducer 合并状态, 用 8 种终止原因精确刻画结束。四个入口共享同一个生成器驱动的闭环。下一章我们看流动策略如何决定 “每步激活几个能力”。

第 6 章

流动策略：每步激活多少能力

FlowPolicy 是 Fullspace 与图框架的分水岭。它决定每一步激活一个能力还是一整个邻域——这是无屏障并行的来源。

6.1 FlowPolicy 抽象

```
class FlowPolicy(ABC):
    @abstractmethod
    def select(self, manifold, query) -> list[Hit]: ...
    def reset(self) -> None: ... # 默认 no-op, run 入口会调
```

清单 6-1 FlowPolicy 抽象基类

三种内置策略只在 select 返回多少个 Hit 上有差异。它们共用同一个闭环，区别仅在于首步 select 和 _route_next 的分流。

6.2 DiscreteFlow：图等价

DiscreteFlow 每步只激活最近的一个能力，等价于传统的图执行：

```
class DiscreteFlow(FlowPolicy):
    def select(self, manifold, query) -> list[Hit]:
        return manifold.nearest(query, k=1)[:1]
```

清单 6-2 DiscreteFlow：每步恰好 1 个

离散图是流形的退化情况

Fullspace 自我定位为 LangGraph 的严格超集（见 fullspace/_init_.py:7-8）：任何离散图都能在 manifold 上表达（DiscreteFlow + 硬 goto），但 manifold 还能表达图无法表达的连续语义结构。

6.3 FieldFlow：邻域与无屏障并行

FieldFlow

每步激活固定

k

个邻居，多个能力在一步内同时被激活、合并更新、加权合成下一 query——这就是“无屏障并行”：

```
class FieldFlow(FlowPolicy):
    def __init__(self, width: int = 3, min_score: float = 0.0): ...
    def select(self, manifold, query) -> list[Hit]:
        hits = manifold.nearest(query, k=self.width)
        hits = [h for h in hits if h.score >= self.min_score]
        return hits or manifold.nearest(query, k=self.width) # 不 stall 兜底
```

清单 6-3 FieldFlow：邻域扩散

无屏障 vs superstep

LangGraph 的并行在每个 superstep 末尾有 barrier，等所有并发分支汇合才进入下一步。FieldFlow 在同一步内激活多个能力，它们的 updates 在本步内合并，intent 加权合成下一 query——没有 superstep barrier，延迟更短。

6.4 WavefrontFlow: widening 邻域

WavefrontFlow

每步激活的邻域随步数

t

递增，像波一样从起点向外扩散：

```
class WavefrontFlow(FlowPolicy):
    def __init__(self, base_width=2, growth=1, max_width=None): ...
    def reset(self): self._t = 0
    def select(self, manifold, query) -> list[Hit]:
        self._t += 1
        k = self.base_width + (self._t - 1) * self.growth
        if self.max_width: k = min(k, self.max_width)
        return manifold.nearest(query, k=k)
```

清单 6-4 WavefrontFlow: 扩散探索

6.5 _route_next 的分流

下一步路由按 flow 类型分流，这是几何路由相对边连线的根本优势：

```
if isinstance(self.flow, DiscreteFlow):
    intent = max(intents, key=lambda x: x[1])[0] # 取分数最高的 intent
    decision = self.router.route(intent)         # 走混合路由器
    return [Hit(decision.capability, decision.score)]
else: # field / wavefront
    query_vec = self._combine_intents(intents)    # 多 intent 加权平均
    hits = self.flow.select(self.manifold, query_vec) # 再用流策略选
    return hits
```

清单 6-5 _route_next 两条路径

多意图可被几何合并

DiscreteFlow 走“单 intent + Router”；FieldFlow/WavefrontFlow 走“多 intent 加权 + flow.select”，完全绕开 Router。多意图可同时被几何合并，无需手工编排扇出/扇入——这是几何路由的根本优势。

6.6 小结

FlowPolicy 决定每步激活多少能力：DiscreteFlow 每步 1 个（图等价）、FieldFlow 每步固定 k 个（无屏障并行）、WavefrontFlow 每步递增（扩散探索）。_route_next 在离散与非离散之间分流，让多意图得以几何合并。下一章我们深入混合路由器。

第 7 章

混合路由器：粗跳 + 消歧 + 物化

Router.route 是 Fullspace 延迟优势的核心。它默认只做一次 ANN 查询，仅在真正歧义时才请 LLM 消歧，在近失配时物化新能力。

7.1 三层结构

混合路由器有三层判定，层层递进，前一层满足就不进下一层：

1. intent 为 None → 返回 None (no_route)
2. 查 top-2 邻居 hits
3. 默认 chosen = hits[0]
4. 歧义判定：top-1 与 top-2 分差 $< \text{margin}$ → 调 disambiguator (LLL 仅在此)
5. 亲和剪枝：chosen.score $\geq \text{threshold}$ → 直接返回（不调 LLM）
6. 近失配物化：materializer 存在 → 造新能力并注册
7. 兜底：best-effort 返回 top-1

清单 7-1 Router.route 判定流程

7.2 亲和力裁剪

第一层是亲和力裁枝——一次最近邻查询替代 N 次条件评估。当最近邻的 $\text{cosine} \geq \text{threshold}$ (默认 0.3)，直接命中，不进入更复杂逻辑。这是 Fullspace 相对 LangGraph 的延迟轴胜利。

threshold 与 affinity 的区别

threshold 用的是 cosine $[-1,1]$ ，affinity 是把 cosine 线性映射到 $[0,1]$ 的 $(\text{score}+1)/2$ 。threshold=0.3 对应 affinity ≈ 0.65 。这是常见混淆点。

7.3 歧义消解：何时请 LLM

第二层只在 top-1 与 top-2 太接近时 (分差 $< \text{margin}$, 默认 0.15) 才调 disambiguator。LLM 的角色从“每步必调”降级到“罕见兜底”：

```
if (disambiguator is not None
    and second is not None
    and (top.score - second.score) < margin):
    picked_id = disambiguator(intent, hits) # ← LLM 只在这里被调用
    if picked_id in manifold:
        chosen = Hit(manifold.get(picked_id), top.score)
```

清单 7-2 歧义消解触发条件

7.4 物化：运行时涌现

第三层是 spawn-on-miss：当没有任何能力达到 threshold，而 materializer 存在，就当场造一个新能力并注册。这与第 4 章 manifold.find_or_materialize 的路径 C 同源——

```

m = Manifold(HashEmbedder())
m.register(Capability("greet", "greet the user hello"))
eng = Engine(m, router=Router(
    m, threshold=0.99, # 强制近失配
    materializer=lambda desc, score: Capability("fallback", desc),
))
eng.bind("greet", lambda ctx: NodeResult(intent="zzzzqqqq unmatched gibberish"))
eng.bind("fallback", lambda ctx: NodeResult(halt=True))
r = eng.run("greet the user hello")
assert "fallback" in m # 被物化进流形
assert r.trajectory == ["greet", "fallback"]

```

清单 7-3 物化示例 (来自 tests/test_engine.py)

7.5 embedding 缓存

ReAct 循环里 “act” “observe” 这类意图会反复出现，每次路由 hop 都要重新 embed。CachedEmbedder 用 FIFO 缓存重复文本，对神经模型或 OpenAI API 能省巨大真实收益——循环场景调用数可降 20×。

embedder 是关于文本的纯函数

所以缓存绝对正确，没有失效问题。CachedEmbedder 实现了和 Embedder 一样的接口 (is-a)，任何接受 Embedder 的地方都接受它——这是装饰器模式与依赖倒置的双重示范。

7.6 小结

混合路由器三层递进：亲和力裁剪（默认，一次 ANN）→ 歧义 LLM 消歧（仅 top-2 接近）→ 近失配物化（涌现）。LLM 从每步必调降级为罕见兜底，这是延迟优势的来源；materializer 是涌现的工厂。下一章我们看状态与检查点。

第 8 章

状态、检查点与时间旅行

可恢复、可回放的状态机是“替代而非并列”的前提。这一章讲 Fullspace 如何用每键 reducer、检查点和 trajectory 实现持久化与时间旅行。

8.1 每键 reducer 模型

Fullspace 把每个状态键建模成一个带 reducer 的通道，但实现极简——一个 dict 是 state，另一个 dict 是 spec：

reducer	语义
overwrite	无条件返回 new（默认）
last_value	new is not None 时才覆盖
add	列表拼接，支持标量起步

表 8-1 三个内置 reducer

last_value 的妙用

`last_value(prev, new) = new` if new is not None else prev。None 表示“这一步没更新这个键，保持原值”。这是“可选更新”的常用模式——节点可以只改它关心的字段。

8.2 Checkpoint: 存什么

```
@dataclass
class Checkpoint:
    checkpoint_id: str          # f"{thread_id}:{step:04d}"
    thread_id: str
    step: int
    state: dict                 # 完整状态快照（深拷贝）
    trajectory: list[str]
    step_groups: list[list[str]]
    parent_id: Optional[str]    # 链表指针，预留分支拓扑
    terminated_by: Optional[str] = None
```

清单 8-1 Checkpoint 数据类

`checkpoint_id` 形如 `job1:0002`，单调按 `step` 递增。`parent_id` 指向同线程上一个检查点，形成 timeline，预留了“从历史检查点分叉”的扩展口。

8.3 InMemory 与 Sqlite

InMemoryCheckpointer

零依赖，进程生命周期存储。SqliteCheckpointer 用标准库 sqlite3，一张表八列，state/trajectory/step_groups 用 JSON 序列化：

```
CREATE TABLE checkpoints (
    thread_id      TEXT,
    checkpoint_id  TEXT,
    step           INTEGER,
    state          TEXT,    -- JSON
    trajectory     TEXT,    -- JSON
    step_groups    TEXT,    -- JSON
    parent_id      TEXT,
    terminated_by  TEXT,
    PRIMARY KEY (thread_id, checkpoint_id)
);
```

清单 8-2 SqliteCheckpoint 的 schema

state 的值必须 JSON-able

SqliteCheckpoint 用 `json.dumps` 序列化 `state`，所以 `state` 里不能放不可序列化的对象（如未绑定的函数）。需要存 handler 引用请用 `id` 字符串，运行时再查表。

8.4 resume 与时间旅行

检查点的写入时机是正确性核心：每步末尾写（继续型） + 中途 advance 写 + 终态写（终止型），三类路径都被覆盖。这意味着每一步都有可恢复的快照：

```
eng = Engine(m, checkpointer=InMemoryCheckpoint())
# 用小预算故意打断
r1 = eng.run("work repeat the processing step",
             state={"n": 5}, thread_id="job1", max_steps=2)
# terminated_by='budget', trajectory=['work', 'work']

hist = eng.history("job1")          # 时间旅行：列出所有检查点
r2 = eng.resume("job1", task="work repeat the processing step",
               max_steps=25)         # 从最新检查点续跑
# terminated_by='sink', trajectory=['work', 'work', 'work', 'work', 'end']
```

清单 8-3 中断与续跑（来自 `examples/interrupt_resume.py`）

时间旅行 = get / list / put

get(thread_id) 回到现在或某一帧；list(thread_id) 列出整条时间线；put 每次落盘。要落盘跨进程，把 InMemoryCheckpoint 换成 SqliteCheckpoint(path="job.db") 即可。

8.5 trajectory: 流形上的空间路径

Fullspace 把“在能力流形上走过哪些点”当作一等状态，和标量 state 一起 checkpoint。所以时间旅行回放的不仅是数据，还有计算在流形上走过的**空间路径**。annotate_positions 给每步填上 3D 坐标，但——

空间是派生视图，不是控制信号

position3d

仅供可视化层填充（annotate_positions），路由绝不读它。控制流永远走 ANN/goto/intent，避免循环依赖。这是“把计算在流形上的轨迹可视化”与“让可视化反过来驱动计算”之间的纪律。

8.6 小结

Fullspace 用每键 reducer（同构但极简于 LangGraph channels）合并状态，用 Checkpoint 持久化每一步，用 InMemory/Sqlite 两种存储，用 resume/history/get_checkpoint 支持恢复与时间旅行。trajectory 把流形上的空间路径也存了下来，但仅供可视化。下一章看 Fullspace 如何与 LangGraph 双向互操作。

第 9 章

LangGraph 双向互操作

要“替代”而非“并列”，互操作是承重墙。这一章讲 Fullspace 如何既吞下 LangGraph 子图，又把自己反向暴露成 LangGraph 节点和 langchain Runnable。

9.1 为什么互操作是承重墙

模块 docstring 直言 interop 是替代 LangGraph 的“load-bearing wall”（承重墙）。它的工程含义是：**移除 interop, engine 仍然完整可用；有了 interop, 整个 LangChain/LangGraph 生态都成为 Fullspace 的外围。**interop 单向依赖 engine, 不改 engine 任何代码。

9.2 LG→FS: 子图作为能力

as_capability 把编译后的 LangGraph 子图变成流形上的一个 region, 返回 (Capability, handler) 二元组：

```
cap, handler = as_capability(
    compiled_subgraph,
    capability_id="retriever",
    description="retrieve and summarize documents",
    goto="writer",          # 跑完跳到 writer
    map_in=lambda s: {"query": s.get("q")},
    map_out=lambda out, s: {"docs": out["summaries"]},
)
m.register(cap); eng.bind("retriever", handler)
```

清单 9-1 as_capability: 把 LG 子图嵌入流形

把子图当汇点

intent 与 goto 同时为 None 时, NodeResult 既无 intent 又无 goto, engine 按 no_intent

终止。也就是说, “把子图当汇点”的写法就是 intent=goto=None。

9.3 FS→LG: engine 作为节点

as_langgraph_node 反向把 engine 变成一个 LangGraph 节点函数。对调用方完全透明——一张大 LangGraph 图里某个节点, 背后其实是整条 Fullspace 流形:

```
node = as_langgraph_node(
    engine,
    task=lambda s: s["task"],          # 从 LG state 动态派生 task
    map_state_out=lambda fs, lg: {"n": fs["n"]},
)
g = StateGraph(dict)
g.add_node("fs_step", node); g.set_entry_point("fs_step")
app = g.compile()
```

清单 9-2 as_langgraph_node: engine 反嵌 LG

9.4 FullspaceRunnable: langchain 契约

FullspaceRunnable 让 engine 能塞进任何接受 langchain Runnable 的位置 (LCEL chain、LangServe、LangGraph 工具节点)。它刻意匹配 LangGraph 编译产物的表面:

Runnable 方法	Engine 方法
invoke(input)	engine.run(task, state)
stream(input)	engine.stream(...) 逐事件 yield
ainvoke(input)	await engine.ainvoke(...)
astream(input)	async for ev in engine.astream(...)

表 9-1 Runnable 方法映射

契约对称性

Engine 本身已有 run/stream/ainvoke/astream; FullspaceRunnable 仅做归一 (_parse_input / _event_to_chunk / _result_to_output), 零状态、零缓冲。所以一个 FS 引擎可以直接用 runnable | ChatOpenAI() 这种 LCEL 表达式串联。

9.5 小结

interop 是替代 LangGraph 的承重墙: as_capability 把 LG 子图变成流形 region (LG→FS), as_langgraph_node 把 engine 变成 LG 节点 (FS→LG), FullspaceRunnable 暴露完整 langchain Runnable 契约。三者都不改 engine 代码。下一章看流式与异步。

第 10 章

流式与异步

对标 LangGraph 的 `stream/astream`，Fullspace 每步产生一个 `StepEvent`，并原生支持 `async def` 节点——为真实 LLM 调用预留的接口，而非空洞承诺。

10.1 StepEvent: 每步一个事件

```
@dataclass
class StepEvent:
    step: int                # 从 1 开始
    group: list[str]         # 本步激活的能力 id
    updates: dict             # 本步原始更新
    state: dict              # 合并后快照
    trajectory: list[str]
    terminated: bool
    terminated_by: Optional[str] = None
```

清单 10-1 StepEvent 字段

10.2 同步流


```
for ev in eng.stream("plan the research steps"):
    print(f" step {ev.step}: activated={ev.group}"
          + (f" ({ev.terminated_by})" if ev.terminated else ""))
```

清单 10-2 同步流式 (来自 examples/streaming.py)

10.3 异步流与 async 节点

异步版还能直接 await async def 节点——真实场景里这里会是 async LLM / HTTP 调用:

```
async def search(ctx):                                # 真实场景: async LLM/HTTP
    await asyncio.sleep(0)
    return NodeResult(updates={"found": "facts (async)"},
                           intent="summarize the findings into key points")

async for ev in eng.astream("plan the research steps"):
    print(f" step {ev.step}: answer={ev.state.get('answer')!r}")
```

清单 10-3 异步节点: 真实 LLM 调用的预留

同一签名同步异步自适应

`_invoke_handler_async` 对 handler 返回值 `inspect.isawaitable`

判定, 需要则 `await`。所以同一个 handler

签名既支持同步也支持异步, 引擎自动适配——这是混用 LLM SDK

同步/异步调用的关键。

10.4 小结

`stream/astream` 每步产出 `StepEvent`, `run/ainvoke` 是其聚合。async def 节点被引擎自动 `await`, 同一签名可同步可异步。这是对标 `LangGraph` `stream/astream` 的完整对等, 且 `FullspaceRunnable` 把同样的表面暴露给 `langchain` 生态。下一章用基准测试验证这些设计。

第 11 章

基准测试：与真实 LangGraph 的诚实对比

评测 harness 是事实来源。这一章讲它如何设计、测出什么，以及如何诚实地读懂结果——包括 Fullspace 在某些轴上其实是输的。

11.1 harness 设计

eval 模块用 5 个量化指标 + 1 个表达力判定，在相同工作流上把 Fullspace 与已安装的真实 LangGraph 直接对比：

指标	含义
success	是否产出预期轨迹/答案
node_executions	节点执行次数（少更优）
routing_calls	路由决策次数（FS=ANN，LG=条件边）
elapsed_ms	墙钟（仅指示性）
deterministic	同输入两次运行轨迹是否一致

表 11-1 评测指标

如何数 ANN 调用

eval 用一个 `_CountingRouter(Router)` 子类，每次 route 自增。Fullspace 的 `routing_calls = 1 + router.count`——那个 1 是起始 ANN 定位，后续每次路由由各加一次。

11.2 六个对照用例

用例	模式	LG 可表达
linear	$A \rightarrow B \rightarrow C$	是
branch	入口随任务路由	是（条件边）
loop	循环 3 次再 end	是（条件边回环）
dynamic_spawn	运行时物化新能力	否
react_loop	2 轮 think-act-observe	是
ood_robustness	OOD 任务	否（报错）

表 11-2 eval 的六个 case

11.3 镜像模式：平手，但路由 FS 输

在纯静态镜像模式（linear/branch/loop/react）上，正确性与节点执行数**平手**。但要注意一个诚实的事实：

静态模式上 Fullspace 路由更多

LangGraph 的预连边成本是 0 次路由决策；Fullspace 每跳多一次 ANN。这是动态/软路由的代价，要等规模上来才被次线性 ANN 翻盘。Fullspace 当前的结构性优势是表达力（dynamic_spawn）与 OOD 鲁棒性。

11.4 scaling：80~120× 的延迟优势

`scaling.py` 是翻转延迟轴的关键实验。它对比 `NumpyAnnIndex` 与 `FaissIndex` 在 $N \in \{1k, 5k, 20k\}$ 的单次查询延迟：

```
def run(sizes=(1000, 5000, 20000), n_queries=500, dim=256):
    for n in sizes:
        ids, vecs, _ = _build_corpus(n, dim)          # N 个能力向量
        queries = _make_queries(vecs, n_queries, dim) # 真实向量+std=0.03 噪声
        t_np = _time_per_query(np_idx, queries)        # warm up 后计时
        t_fa = _time_per_query(faiss_idx, queries)
        kind = "IVFFlat" if n >= faiss_idx.nlist*39 else "FlatIP"
```

清单 11-1 `scaling` 方法（精简）

scaling 五个可信度信号

(1) 单变量对照：同一批向量/查询/dim/k，唯一变量是索引实现；(2) 固定 RNG (`default_rng(0)`) 保证可复现；(3) warm up 排除冷启动；(4) 算法切换阈值透明—— $N=1000$ 用 FlatIP 时与 numpy 同阶，承认差距仅是实现质量；(5) 结论分级——亚线性优势只在过阈值 ($N \geq 3900$) 后才声明。

结论： $N=5k \sim 20k$ 区间，FAISS 比 numpy 快约 $80 \sim 120 \times$ （量级为两个数量级）。小 N 诚实承认同阶。

11.5 如何读懂这份基准

这份基准的叙事是“承认劣势→指出翻转条件→用受控实验佐证”。它在静态模式上诚实承认 Fullspace 路由更多，在表达力与 OOD 上声明胜出，在规模化延迟上用受控实验佐证 $80 \sim 120 \times$ 的量级优势。这种诚实比单纯报速度数字更可信，也更像图灵图书风格的论证。

11.6 小结

eval harness 是事实来源。镜像模式平手（但 FS 路由更多），表达力与 OOD 上 FS 胜（LG 无法表达/会报错），规模化延迟上 FS 胜（FAISS $80 \sim 120 \times$ ）。读基准要先跑它、再下结论。下一章讲如何扩展 Fullspace。

第 12 章

扩展 Fullspace

Fullspace 的每个核心抽象都留了扩展点。这一章用四个最小示例，演示如何插入自己的 embedder、索引、流动策略和检查点器。

12.1 自定义 Embedder

子类 `Embedder`，实现 `embed` 与设置 `dim` 即可。下面是一个用随机投影做演示的 embedder（生产请接真实模型）：

```

from fullspace.manifold.embedding import Embedder

class RandProjEmbedder(Embedder):
    def __init__(self, dim=128):
        self.dim = dim
        # 固定投影矩阵保证可复现
        rng = np.random.default_rng(0)
        self._P = rng.standard_normal((dim, 4096)).astype(np.float32)

    def embed(self, text: str) -> np.ndarray:
        # 把文本哈希成稀疏 bag, 再投影归一化
        v = np.zeros(4096, dtype=np.float32)
        for tok in re.findall(r"[a-z0-9]+", text.lower()):
            v[hash(tok) % 4096] += 1.0
        v = self._P @ v
        n = np.linalg.norm(v)
        return v / n if n > 0 else v

```

清单 12-1 自定义 Embedder

可复现性

务必用固定 seed (如

default_rng(0)) 生成投影矩阵, 否则跨进程向量不一致, 破坏索引。这正是 HashEmbedder 用 md5 而非内建 hash 的原因。

12.2 自定义 ANN 后端

子类

add/search/remove/vector_of/_len_。search
score)] (注意是 id-score 对, 不是 Hit) :

AnnIndex, 实现

返回 list[(id,

```

from fullspace.manifold.index import AnnIndex

class HnswIndex(AnnIndex):
    def __init__(self, dim):
        import hnswlib
        self.dim = dim
        self._idx = hnswlib.Index(space="cosine", dim=dim)
        self._ids = []
        self._dirty = True
    def add(self, id, vector): ...      # 维护 id 内部整数标签映射
    def search(self, query, k=5): ...
    def remove(self, id): ...
    def vector_of(self, id): ...
    def __len__(self): return len(self._ids)

```

清单 12-2 自定义 ANN (接 hnswlib 示意)

12.3 自定义 FlowPolicy

子类 `FlowPolicy`, 实现 `select(manifold, query)` -> `list[Hit]`。下面是一个“只激活 score 最高的前两个”的变体:

```

from fullspace.engine.flow import FlowPolicy

class TopTwoFlow(FlowPolicy):
    def select(self, manifold, query):
        return manifold.nearest(query, k=2)
    # reset 可省略 (无内部状态)

eng = Engine(m, flow=TopTwoFlow())

```

清单 12-3 自定义 FlowPolicy

12.4 自定义 Checkpointer

子类 `Checkpointer`, 实现 `put/get/list` 三方法。下面接 Redis 的示意:


```

from fullspace.state import Checkpointer, Checkpoint

class RedisCheckpointer(Checkpointer):
    def __init__(self, client): self._r = client
    def put(self, cp: Checkpoint): ...      # JSON 序列化存 Redis hash
    def get(self, thread_id, checkpoint_id=None): ...  # 不带 id 取最新
    def list(self, thread_id): ...          # 按 step 升序返回

```

清单 12-4 自定义 Checkpointer

12.5 小结

Fullspace 的四个核心抽象——Embedder、AnnIndex、FlowPolicy、Checkpointer——都只要子类化并实现少数方法即可替换。配合 Manifold/Engine 的构造参数注入，你可以从 HashEmbedder+NumpyAnnIndex 的演示配置，平滑升级到 SentenceTransformersEmbedder+FaissIndex 的生产配置，甚至换成自己的 HNSW、Redis 后端。结合前十一章的原理，你现在已能驾驭整个框架。

附录 A

示例索引

Fullspace 自带的可运行示例，每个教会一个模式。

示例	模式	运行命令
linear_pipeline	$A \rightarrow B \rightarrow C$ (图等价)	<code>python -m fullspace.examples.linear_pipeline</code>
branching	任务相关的软路由	<code>python -m fullspace.examples.branching</code>
react_agent	ReAct 思考-行动-观察循环	<code>python -m fullspace.examples.react_agent</code>
interrupt_resume	检查点/人在回路/续跑	<code>python -m fullspace.examples.interrupt_resume</code>
streaming	同步/异步流式 + async 节点	<code>python -m fullspace.examples.streaming</code>

表 A-1 示例与模式

viz 与 eval 的 CLI: `python -m fullspace.viz` (生成 3D 球 HTML)、`python -m fullspace.eval` (六用例对照表)、`python -m fullspace.eval.scaling` (FAISS scaling)。注意 eval 依赖 langgraph, scaling 在 $N \geq 3900$ 时需要 faiss-cpu。

附录 B

API 速查

最常用的类与方法签名一览（基于 0.1.0 源码）。

```
Capability(id, description, metadata={}, vector=None)
Manifold(embedder, index=None, projector=None)
    .register(cap) / .register_many(caps) / .remove(id)
    .nearest(query, k=5) -> list[Hit]      # query: str | ndarray
    .find_or_materialize(query, threshold=0.5, k=1, materializer=None)
    .get(id) / .vector_of(id) / .project(id) / .project_all()
HashEmbedder(dim=256); CachedEmbedder(inner, maxsize=4096)
NumpyAnnIndex(dim); FaissIndex(dim, nlist=100, nprobe=10)
PCAPProjector(); UMAPProjector(n_neighbors=15, random_state=0)
```

清单 B-1 manifold 核心

```

Engine(manifold, flow=None, router=None, terminator=None,
      handlers=None, max_steps=None, state_spec=None, checkpointer=None)
  .bind(id, handler) / .bind_many({id: handler})
  .run(task, state=None, thread_id=None, max_steps=None) -> RunResult
  .stream(...) -> Iterator[StepEvent]
  async .ainvoke(...) / async .astream(...)
  .resume(thread_id, task, max_steps=None) / async .aresume(...)
  .history(thread_id) -> list[Checkpoint]
  .get_checkpoint(thread_id, checkpoint_id=None)
NodeResult(updates={}, intent=None, goto=None, halt=False)
Router(manifold, threshold=0.3, margin=0.15, disambiguator=None, materializer=None)
DiscreteFlow(); FieldFlow(width=3, min_score=0.0); WavefrontFlow(base_width=2, growth=1)

```

清单 B-2 engine 核心

```

merge_updates(state, updates, spec=None)
overwrite / last_value / add      # 内置 reducer
InMemoryCheckpoint(); SqliteCheckpoint(path=None)
as_capability(app, capability_id, description, *, intent=None, goto=None,
              map_in=None, map_out=None) -> (Capability, handler)
as_langgraph_node(engine, task, *, map_state_out=None) -> node_fn
FullspaceRunnable(engine)

```

清单 B-3 state 与 interop 核心

附录 C

术语表

本书出现的关键术语速查。

术语	含义
能力流形	所有能力描述嵌入成的高维度量空间
能力空间路由	用最近邻查询定位下一步能力（取代边连线）
Capability	流形上的一个点（id+description+metadata+vector）
sink	metadata["sink"]=True 的能力，命中即终止
intent	软路由指令：描述“下一步想做什么”，ANN 找最近邻
goto	硬路由指令：精确跳指定 capability id
halt	强制本步后终止
affinity pruning	亲和力裁剪：score $\geq$ threshold 直接命中
spawn-on-miss	近失配物化：无匹配时造新能力并注册
disambiguator	歧义消解回调（通常接 LLM），仅 top-2 接近时调用
materializer	物化回调：负责构造新 Capability
FlowPolicy	流动策略：决定每步激活几个能力
reducer	每键状态合并函数（overwrite/last_value/add）
Checkpoint	单步状态快照（含 state/trajectory/parent_id）
时间旅行	用 history/get_checkpoint 读取或分叉历史帧
承重墙	interop：移除不影响 engine，有则接入整个 LG 生态

表 C-1 术语表
